Research report

iPRA with parameter witnesses

A finite witness interface for a complete intuitionistic arithmetic calculus

Prepared for Vladimir Reshetnikov

12 November 2026

Abstract

The current ConciseIPRA development asks whether its paired introduction rule needs arbitrary numerical witness terms. We prove that it does not. First, zero and numerical variables suffice. A protected proof translation then restricts the variable to the newest ambient parameter. The final presentation uses only one bit: it selects zero or that parameter, with zero as the empty-scope fallback. The language remains raw scalar iterator terms and equations with the paired binder D. A clause presentation has five primitive rule families and no separately postulated logical axioms.

The critical arithmetic lemma is a predecessor construction using only computation, equality and literal-equation induction. Unconditional Boolean identities remove a circular appeal to numerical case distinction under an induction hypothesis. A paired formula then names every term by a fresh variable, recovering every omitted introduction. The resulting calculus proves exactly the same sequents as the pinned baseline and inherits both whole-domain interpretation composites with full first-order intuitionistic primitive recursive arithmetic.

A matched, standalone, publicly scoped JSON proof checker decreases from 2,956 to 2,947 normalized bytes, from 837 to 833 significant tokens, and from 977 to 971 AST nodes. This modest implementation reduction accompanies a substantive restriction of the primitive witness interface. It is not a claim to beat the repository's 516-byte compact codec, nor a proof of absolute minimality. The new mathematical result is a written proof; the accompanying executable checks are not a Lean formalization.

Baseline pinned to ProveIt commit `9f9545f5ffd9e0392d4a36639503aca12383450a`.

Contents

1 Result and comparison target **1**

2 Complete definition of the proposed calculus **1**

2.1 Numerical terms, formulas and scopes . . . 1

2.2 Five primitive rule families . . . 2

2.3 What is derived . . . 3

3 Why arbitrary witness terms can be removed **3**

3.1 The essential naming formula . . . 3

3.2 The arithmetic dependency that prevents circularity . . . 4

3.3 Why the newest variable needs a protected proof . . . 4

4 Full intuitionistic bi-interpretability **5**

4.1 The target is full first-order iPRA . . . 5

4.2 The ordinary realization and falsity . . . 5

4.3 Both composites, rather than equiconsistency alone . . . 6

5 Executable comparison and validation **6**

5.1 A matched complete checker . . . 6

5.2 The Boolean field is a decoder, not a term . . . 7

5.3 What the executable evidence establishes . . . 7

6 Scope of the achievement **7**

A The restricted-witness theorem **8**

A.1 Seven primitive rule patterns . . . 8

A.2 Unrestricted case introduction for a closure . . . 9

A.3 Full formula transport without term substitution on proofs . . . 9

A.4 Naming every term with a fresh variable . . . 10

A.5 Recovery of every omitted introduction . . . 11

B Restriction to zero or the newest parameter **12**

B.1 Hereditary derivability and uniform cut . . . 12

B.2 The logical and equational interfaces at the hereditary level . . . 12

B.3 A hereditary naming packet . . . 13

B.4 Translation of the native kernel . . . 13

B.5 A total two-choice interface . . . 14

C Direct transfer to the five-rule clause checker **15**
C.1 The clause interface . . . 15
C.2 Hereditary derivations with clause contexts . . . 15
C.3 The packet and the completeness induction . . . 16

D A predecessor from equations and equation induction alone **17**
D.1 Resources and a finite-value lemma . . . 17
D.2 Addition and parity identities . . . 18
D.3 A three-state update and a ramp . . . 19
D.4 The factor map . . . 20
D.5 A comparator . . . 21
D.6 Saturation and predecessor . . . 21
D.7 Provenance and verification status . . . 23

E Archive map and reproduction **23**

1 Result and comparison target

The proposed theory is the five-family calculus $\mathcal{B}$ of Section 2. Its introduction instruction contains *one bit instead of an arbitrary term.* All arithmetic terms, ordinary numerical domains, equality, and the reading of the logical binder are unchanged. This is a restriction of the primitive rule interface, not a change to the set of mathematical consequences.

The comparison is the repository at the full commit printed on the title page, inspected on 12 November 2026. The main README records seventy-nine research packages and several previously established reductions. In particular, raw clauses, five rule families, computational equality and two-case evaluation are already present. They are reused here and are not new claims. The README's further question 2 asks whether introduction at zero or at a variable suffices, and identifies the recovery of predecessor without illicit substitution as an obstruction [1, 2]. The first theorem below answers that question; the strongest version keeps only zero and the newest variable.

Theorem 1.1 (Main result)**.** *For every finite numerical scope, every well-scoped context of native clauses* Γ*, and every native Eq/D formula* A*,*

$$\Gamma \vdash_{\mathcal{B}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}} A,$$

where $\mathcal{C}$ *is the baseline raw clause calculus with unrestricted witnesses. Consequently the ordinary realization of* $\mathcal{B}$ *is bi-interpretable and equiconsistent with full first-order* iPRA*. Both numerical domains are whole and both interpretation composites have the identity graph* $x = y$*.*

The proof is constructive at the metalevel: a finite source derivation can be translated to a finite target derivation. It does not establish that the new certificates are shorter; the naming construction generally makes them longer. The resource being reduced is the trusted rule interface and its matched checker, not certificate length or checking time.

There is no language-independent smallest checker without a fixed encoding and implementation model. Accordingly this report establishes an explicit strict matched reduction and a mathematical restriction. It does not infer an absolute lower bound from a collection of prior drafts.

2 Complete definition of the proposed calculus

2.1 Numerical terms, formulas and scopes

Terms have one numerical sort:

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the last child h binds u. The counter n and seed a do not bind anything. Every occurrence of I is a finite term description; there is no object-language variable for a function or its code. Substitution is ordinary capture-avoiding substitution. Terms are raw: no normalization, conversion oracle or evaluator is hidden in equality of syntax trees.

The numerical scope is a finite ordered list $x_1, \ldots, x_k$ of available free parameters. Local eigenparameters are appended to this list. Define

$$\ell_k = \begin{cases} x_k, & k > 0, \\ 0, & k = 0. \end{cases}$$

This convention concerns proof syntax; it introduces no new numerical constant or interpretation-domain restriction.

Formulas and clauses are

$$A ::= (s = t) \mid D_{x,y}A, \qquad \langle A;\, y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

The paired binder binds both x and y in its matrix. A context is a finite list of reusable clauses. An ordinary temporary assumption F abbreviates the constant clause $\langle 0 = 0;\, v.F\rangle$, with v unused. An empty context contains no implicit axiom clauses.

The ordinary meanings, used in proving adequacy rather than by the checker, are

$$\begin{aligned}(s = t)^\circ &= (s = t),\\ (D_{x,y}B)^\circ &= \exists x(B^\circ(x,0) \to \forall y\, B^\circ(x,y)),\\ \langle A;\, y.B(y)\rangle^\circ &= A^\circ \to \forall y\, B^\circ(y).\end{aligned}$$

The designated native contradiction is $0 = S0$. Ordinary falsity is discussed explicitly in Section 4.

2.2 Five primitive rule families

All premises retain the indicated context. These are the entire primitive inventory.

1. Reflexivity. For every in-scope term t,

$$\frac{}{\Gamma \vdash t = t}. \tag{Refl}$$

2. Evaluation. From equations $s = t$ and $b = r$, infer $r = \Phi$, in exactly either of these two syntactic cases:

b	s	Φ
$I(0, a; u.h)$	a	t
$I(Sc, a; u.h)$	$I(c, a; u.h)$	$h[t/u]$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash b = r}{\Gamma \vdash r = \Phi}. \tag{Eval}$$

All equalities in the case conditions are literal syntax equality. There is no third case for $b = s$. The familiar iterator computation equations and equality transport are derived, as explained below.

3. Clause use. For an in-scope term t,

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad (\langle A;\, y.B(y)\rangle \in \Gamma). \tag{Use}$$

Clause specialization retains arbitrary terms. Restricting specialization as well as introduction is not part of this result.

4. Paired elimination.

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, \langle B(b,0);\, y.B(b,y)\rangle \vdash A}{\Gamma \vdash A}. \tag{Elim}$$

Here b is fresh for Γ, A and all free parameters of B. It is appended to the numerical scope in the second premise. The displayed clause is discharged.

5. Introduction with one bit. Choose $\beta \in \{0,1\}$ and set

$$t_\beta = \begin{cases} 0, & \beta = 0, \\ \ell_k, & \beta = 1, \end{cases} \qquad \eta = \begin{cases} z, & \beta = 0 \text{ and } B \text{ is a literal equation}, \\ 0, & \text{otherwise.} \end{cases}$$

Then

$$\frac{\Gamma, B(t_\beta, \eta) \vdash B(t_\beta, Sz)}{\Gamma \vdash D_{x,y} B(x,y)}. \tag{Bit-DI}$$

The eigenparameter z is fresh for the previous scope, context, witness and matrix parameters. The witness is selected *before* z is introduced. In particular z itself can never serve as this witness. The temporary formula assumption is the constant clause defined above. At an empty numerical scope, bit 1 means a case introduction at zero; bit 0 at an equation means progress at zero. The two bits therefore still have different roles there.

2.3 What is derived

There are no further primitive logical axioms, successor axioms, computation axioms, equality schemas, or induction schemas. Hypothesis use is Refl at 0 followed by Use of the constant assumption clause.

Eval with reflexive premises gives

$$I(0, a; u.h) = a, \qquad I(Sc, a; u.h) = h[I(c, a; u.h)/u].$$

The first evaluation case also derives the root composition rule $s = t,\ s = r \vdash r = t$. To see this explicitly, put $J = I(0, s; u.u)$. Evaluate $s = r$ against $J = J$ to get $J = r$; then evaluate $s = t$ against $J = r$ to get $r = t$. Consequently symmetry and transitivity are derived.

For a term template $h(v)$, let $J = I(0, s; u.h(u))$ and $K = I(S0, s; u.h(u))$. Zero evaluation derives $J = t$ from $s = t$; successor evaluation with $K = K$ derives $K = h(t)$. The same calculation with $s = s$ gives $K = h(s)$, so root composition gives $h(s) = h(t)$. Thus term congruence and crossed transport follow without a separate transport instruction. These are the established two-case evaluation reductions [3, 4]; they are repeated to make the rule inventory self-contained.

The rest of the proof derives closure introduction and elimination, full formula transport, equation induction, predecessor and arbitrary witness introduction. The last derivation is the new restriction result. Ordinary intuitionistic connectives are definable, and their ordinary rules are recovered through the already established native interpretation. They are not built into the checker.

3 Why arbitrary witness terms can be removed

3.1 The essential naming formula

Put

$$C(v, a, b) = I(v, a; u.b), \qquad T(v) = D_{x,y}(C(y, v, C(v, 0, Sx)) = v). \tag{Packet}$$

The variable u is unused in the definition of C. The two useful rows of the matrix are

$$y = 0 : \quad v = v, \qquad y = Sw : \quad C(v, 0, Sx) = v,$$

up to the displayed computation equations.

The zero instance $T(0)$ is introduced with bit 0. Its matrix is an equation, so this is progress; its successor row reduces to $0 = 0$, independently of its local hypothesis. The successor instance $T(Sa)$ is introduced with bit 1 while a is the newest parameter. Its successor row reduces to $Sa = Sa$. Thus an outer closure proves $T(v)$ for every term v using clause specialization, which remains unrestricted.

Open $T(St)$ at a fresh b. Its zero row is a theorem. Specializing the retained clause at $1 = S0$ gives

$$C(1, St, C(St, 0, Sb)) = St,$$

and computation gives $Sb = St$. A predecessor term P satisfying $P(Sv) = v$ then gives $b = t$. Hence any proof needing an arbitrary term t can use a fresh numerical name b, transport along $b = t$, and discharge that name.

This addresses the concrete test case in the README: the ordinary existential assertion $\exists x\,(x = S0)$ is recoverable. Name $S0$ by opening $T(S(S0))$, infer $b = S0$ by predecessor, introduce the existential encoding at the available variable b, and discharge the packet. No substitution of $S0$ for a variable in an unverified restricted derivation is assumed.

3.2 The arithmetic dependency that prevents circularity

It would be circular to obtain predecessor from full iPRA bi-interpretability and then use predecessor to prove that the restricted theory has full iPRA strength. Appendix D therefore supplies an internal predecessor construction from computation, equality and literal-equation induction alone.

The scalar iterator has only a state argument in its update, so predecessor is not an immediate computation. The inherited construction uses parity, a three-state cycle, a ramp, a factor map, a comparator and a saturating counter. The new dependency refinement is important: numerical case splits under an induction hypothesis are replaced by unconditional equations proved in advance.

For example, let N be the parity iterator, j the three-state update, M its orbit, and G the ramp, as defined in the appendix. Before inducting on $M(G(n)) = N(n)$, prove for $b = N(v)$ the identities

$$\begin{aligned} M(g + b + 1) &= C\big(b, j(M(g)), j(j(M(g)))\big), \\ C\big(b, j(b), j(j(b))\big) &= Z(b). \end{aligned}$$

Each is obtained from its two direct computations at $b = 0, 1$ by equation induction. The ramp step then uses the induction hypothesis only as an equality, never as a side assumption in a Boolean case split. This is the precise repair that makes predecessor available before unrestricted witness introduction.

3.3 Why the newest variable needs a protected proof

With zero and arbitrary variables allowed, injective renaming, weakening and cut are proved directly by induction on derivations. Appendix A gives this simpler proof in full.

Restricting the variable to the newest parameter changes that situation: inserting another parameter may make a previously permitted witness cease to be the newest. Thus one cannot silently shift a target proof under a binder. The proof for the final bit calculus instead maintains *hereditary derivability*: it constructs a derivation under every injective renaming and every allowed extension of the retained context. Under a new eigenparameter, a premise is regenerated from this invariant. It is not obtained by a forbidden renaming of an existing target tree.

At the final step of arbitrary witness introduction, the packet opens the name b. It is now the newest parameter, so bit 1 introduces at exactly that name. The step eigenparameter is added only after this selection. The protected translation and its direct clausal version are supplied in the proof appendices. This ordering is also exercised by the eigenvariable-capture rejection test.

4 Full intuitionistic bi-interpretability

4.1 The target is full first-order iPRA

Here iPRA means first-order intuitionistic predicate logic with equality, zero and successor, a function symbol for every fixed finite primitive-recursive description with its defining equations, the usual successor separation and injectivity axioms, and contextual induction for quantifier-free formulas. The claim is not restricted to closed equations, a classical fragment, or a collection of numerical sentences.

The baseline proves native-to-iPRA and iPRA-to-native translations for all formulas and contexts, together with both formula round trips. The pinned Lean endpoint explicitly states `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip` and `native_inconsistent_iff` [5]. These source files were inspected; the baseline Lean build was not rerun in this workspace. Our new proof is not included in that existing formalization.

Theorem 1.1 transfers this exact result because it identifies derivability on the unchanged native formulas, with unchanged numerical terms and clauses. It does not merely show agreement in the standard natural-number model.

4.2 The ordinary realization and falsity

For clarity, elaborate each fixed iterator body $h(u, \vec{p})$ into an ordinary function symbol $i_h(n, a, \vec{p})$, with its two defining equations. Bodies are introduced in finite stages. Let T_I be ordinary intuitionistic first-order logic with actual equality, these symbols and equations, induction for literal equations, numerical coverage

$$\forall x\,(x = 0 \vee \exists y\, x = Sy),$$

and the falsity bridge $(0 = S0) \to \bot$. The native clauses and D are read by the formulas in Section 2. Coverage is recovered by the native logical interpretation. The falsity bridge is explicit in the ordinary realization and expresses the reading of the designated contradiction; it is not asserted to follow from positive iterator equations alone. Neither is an additional instruction in the five-family checker. Stating the bridge in T_I prevents a mistaken appeal to a singleton model of purely positive equations.

The forward arithmetic interpretation sends each scalar iterator to its primitive-recursive iteration in iPRA. The reverse interpretation first derives predecessor, a reflecting equality test, and quantifier-free induction. It then defines pairing with projections and compiles every fixed primitive recursion. For

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

the compiler stores the counter and current value in a pair:

$$\begin{aligned} V(n, \vec{p}) &= I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))), \\ f^*(n, \vec{p}) &= H(V(n, \vec{p})). \end{aligned}$$

Induction proves $L(V(n,\vec{p})) = n$, and the projection equations give the defining equations for f^*. Pairing is a derived construction; it does not enlarge the proof-checking kernel.

4.3 Both composites, rather than equiconsistency alone

For each fixed description, its double translation satisfies the same defining equations. Equation induction proves uniqueness of an iterator with a fixed seed and update; the analogous argument proves uniqueness for primitive recursion. External induction on a finite function description therefore identifies its double translation internally.

Both domains are given by $x = x$, equality is actual equality, and $x = y$ defines a provably total bijection preserving every fixed operation in both composites. The inherited logical macro theorems give the formula composites, including implication, disjunction, conjunction, both quantifiers and falsity. The two translations preserve contradiction, so effective translations of finite proofs establish equiconsistency. No self-consistency statement is asserted.

The unchanged written bridge, equality-reflection and primitive-recursion sources are included in `reference/`, with their license and pinned provenance. The present report proves the new restriction rather than claiming those inherited constructions as new.

5 Executable comparison and validation

5.1 A matched complete checker

The final module reads one JSON certificate from standard input. It validates the closed public formula, term and formula scopes, rule arities, premises and every eigenvariable boundary. It imports only Python's standard JSON and system modules. It contains no arithmetic axiom table, proof database, normalizer, or external theorem prover. Successful acceptance is exit status zero together with the line `ACCEPT`; malformed or incorrect certificates are rejected. As with an ordinary implementation of a proof calculus, practical recursion and memory limits can reject a valid certificate that exceeds the host's resources. The equivalence theorem concerns finite formal derivations, with no fixed machine resource bound.

The baseline is the repository's existing two-case raw clause checker. Both files use the same representation, same public-goal validation, same evaluator, same clause handling and same normalization for size measurement. The change replaces the term-valued introduction field by a Boolean selector and removes its recursive term-validity check.

Complete standalone module	Tokens	AST nodes	Normalized bytes
Pinned two-case control	837	977	2,956
Bit-witness proposal	833	971	2,947
Decrease	4	6	9

The source is normalized with `ast.unparse` under CPython 3.12.14; the exact scripts and results are included. Only the module docstring is removed before normalization, consistently for both programs. Comments, documentation and file formatting are not being used to manufacture the difference. The byte decrease is about 0.30%; token and AST decreases are about 0.48% and 0.61%.

The mathematical interface change is larger than these implementation percentages suggest because the same generic term validator remains necessary for clause specialization and other rule data.

This comparison does not beat every coordinate of every implementation in the repository. Its compact 516-byte program is a different codec profile; the README also records smaller separate token and AST minima. We do not compare a fully scoped public checker against a compact codec as if the contracts were identical. There is no global-minimum claim.

5.2 The Boolean field is a decoder, not a term

Canonical certificates use the integers 0 and 1 in the introduction field. The provided checker defines a total Boolean projection of a JSON value by Python truthiness: false, null, numerical zero, and empty strings, arrays or objects select bit 0; other parsed values select bit 1. This is a serialization choice stated as part of the checker contract. The field is never inserted as a term. The selected numerical term is constructed internally as zero or the newest scoped free-variable level.

Thus acceptance of a noncanonical truthy mode value cannot create a new arithmetic witness. An adapter that insists on literal 0 or 1 can be placed outside this trusted decoder, but no such adapter is silently excluded from the reported measurement. The measured module itself is the full specified checker.

5.3 What the executable evidence establishes

The package contains actual certificates for the zero naming packet, its closure, and its instances at 1 and 2. These are checked by the final restricted checker. Negative cases exercise ill-scoped terms, incorrect computations, malformed formulas and illicit use of the step eigenparameter as the introduction witness. The latter is a material risk: allowing the witness to depend on the new step parameter can validate a false paired assertion. The final checker constructs the witness from the preceding scope, so this dependency is excluded.

An independent scalar evaluator also checks 447 finite equalities from the arithmetic appendix directly from the iterator definitions, including $P(0), \ldots, P(7) = 0, 0, 1, 2, 3, 4, 5, 6$. These computations help catch indexing mistakes. They do not replace the universal equational proof, prove soundness on nonstandard models, or mechanically certify bi-interpretability. The validation logs distinguish executed checks from written theorems.

6 Scope of the achievement

The new result is the finite witness interface and the proof that it preserves full contextual derivability. The arithmetic term language, the five-family clausal organization, two-case Eval, and the bridge to iPRA are inherited and explicitly attributed. The proof also supplies the missing equational dependency refinement for predecessor.

The implementation result is a small strict reduction of a matched complete checker. The artifact does not prove an absolute minimum, establish a new compact-codec record, or provide a newly compiled Lean formalization. It provides a concrete simpler rule interface, written proofs of adequacy and both interpretation composites, executable certificates, exact source provenance and reproducible measurements.

A The restricted-witness theorem

This section uses raw iterator terms, with ordinary capture-avoiding substitution. No computation is built into the syntax. Write

$$C(v,a,b) := I(v,a;u.b), \qquad 1 := S0,$$

where the bound variable u does not occur in b. Thus $C(0,a,b) = a$ and $C(Sv,a,b) = b$ are derivable computation equations. Formulas are equations, Q_yA, and $D_{x,y}A$, with their unchanged ordinary readings

$$Q_yA = A(0) \to \forall y\, A(y), \qquad D_{x,y}A = \exists x(A(x,0) \to \forall y\, A(x,y)).$$

The distinction between a variable and the numerical constant 0 is syntactic throughout.

A.1 Seven primitive rule patterns

Let $\mathcal{R}$ have Hyp, the two iterator computation equations, the one-term crossed transport rule

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = r}{\Gamma \vdash r = h(t)},$$

and the usual two eliminations

$$\frac{\Gamma \vdash Q_yA(y) \qquad \Gamma \vdash A(0)}{\Gamma \vdash A(t)}, \qquad \frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, Q_yB(b,y) \vdash F}{\Gamma \vdash F}.$$

In the second rule b is fresh in the conclusion and the undischarged context. Its one introduction rule is

$$\frac{\Gamma, B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \text{ is the literal } 0, \\ 0, & \text{otherwise,} \end{cases} \tag{RDI}$$

where z is fresh and the supplied witness t must be either the literal 0 or a variable. In particular, every variable witness selects cases. There is no separate rule for introducing Q, and no general term substitution rule on proofs.

Lemma A.1 (Structural rules available before completeness)**.** *Context weakening, injective renaming of numerical variables, and cut are admissible in $\mathcal{R}$.*

Proof. Each assertion follows by induction on the given derivation. A numerical renaming sends a variable witness to a variable witness and fixes the literal 0; it neither changes the root constructor of a formula nor the introduction mode. Renamings are extended under each binder by choosing a fresh variable. Context weakening does not change a witness. In the cut induction the proof replacing a discharged hypothesis is weakened and injectively renamed under eigenvariables; all other rule data are unchanged. This argument does not assert closure under substitution of an arbitrary term for a numerical variable. That property will follow only after unrestricted introduction has been recovered. □

Reflexivity and the usual equational rules are already derived. For example, transport $I(0,a;u.u) = a$ along itself, with the identity template, to obtain $a = a$. Applying crossed transport with major premise $s = s$, minor premise $s = t$, and the identity template gives $t = s$. Transitivity and congruence follow from crossed transport and reflexivity. These constructions need no logical introduction.

A.2 Unrestricted case introduction for a closure

Lemma A.2 (Closure introduction)**.** *Both of the following rules are admissible:*

$$\frac{\Gamma, A(0) \vdash A(Sz)}{\Gamma \vdash Q_y A(y)}, \qquad \frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash Q_y E(y)}, \tag{QC, QP}$$

where z is fresh and E is a literal equation. The first rule permits an arbitrary native formula A, including an equation.

Proof. For QP use RDI at witness 0 on $D_{x,y}E(y)$, with x unused, and then eliminate this dummy witness. This leaves exactly $Q_y E(y)$.

For QC first note the fixed theorem

$$R := D_{r,w}(0 = 0).$$

RDI at witness 0 proves R: its progressive premise concludes the reflexive equation $0 = 0$. Open R at a fresh variable b. In that context introduce $D_{x,y}A(y)$ at witness b. This is case mode even when A is an equation, and the required premise is precisely the given premise, injectively renamed and weakened. Eliminate the dummy witness x to obtain $Q_y A(y)$, and then discharge the opening of R; neither b nor the unused assumption $Q_w(0 = 0)$ occurs in the result. Thus QC does not use a forbidden case introduction of an equation at 0, even when the original numerical context is empty. □

Consequently, ordinary induction for a literal equation follows by QP and closure elimination. All arguments in the separate predecessor construction can therefore be performed inside $\mathcal{R}$ once their equational derivations have been established.

A.3 Full formula transport without term substitution on proofs

Lemma A.3 (Leibniz transport)**.** *For every native formula template $H(v)$, the rule*

$$\Gamma \vdash s = t, \quad \Gamma \vdash H(s) \quad \Longrightarrow \quad \Gamma \vdash H(t) \tag{L}$$

is admissible in $\mathcal{R}$.

Proof. Induct on the number of formula binders in H. State the induction hypothesis uniformly for every ambient context and every assignment of terms to the other free variables. This uniform statement is essential: we apply the induction hypothesis to newly formed instances, rather than substitute terms into previously constructed derivations.

For $H(v) = (a(v) = c(v))$, congruence transports both sides and transitivity joins the resulting equations.

Suppose $H(v) = Q_y A(v, y)$. Apply QC to the target closure. Under its temporary assumption $A(t, 0)$, the reverse equation $t = s$ and the induction hypothesis give $A(s, 0)$. Eliminate the available closure $Q_y A(s, y)$ at Sz to obtain $A(s, Sz)$. The forward equation $s = t$ and the induction hypothesis give $A(t, Sz)$. This proves the QC premise. This explicitly handles the contravariant occurrence of the zero row in the ordinary meaning of Q.

Suppose $H(v) = D_{x,y}B(v, x, y)$. Open the given $H(s)$ at a fresh variable b, obtaining $Q_y B(s, b, y)$. Introduce the target $H(t)$ at the permitted variable witness b. Its case premise assumes $B(t, b, 0)$.

Transport that assumption backwards to $B(s, b, 0)$, eliminate the opened closure at Sz, and transport the result forwards to $B(t, b, Sz)$. All three formulas have fewer binders than H. RDI at b now proves $H(t)$, and the outer elimination discharges b.

The equality premise and other existing derivations are only weakened or injectively renamed when entering a binder. No application of an unproved term-substitution principle occurs. □

A.4 Naming every term with a fresh variable

Assume the separately proved equational predecessor lemma: there is a raw iterator term $p(v)$ such that

$$\mathcal{R} \vdash p(Sv) = v. \tag{P}$$

Then successor injectivity is derived by congruence with p followed by the two instances of (P). This step must be justified before the following argument; importing predecessor from the full iPRA bi-interpretation would be circular.

Put

$$A_v(x, y) := \big(C(y, v, C(v, 0, Sx)) = v\big), \qquad T(v) := D_{x,y} A_v(x, y).$$

The defining rows have the derived equations

$$A_v(x, 0) \iff v = v, \qquad A_v(x, Sw) \iff C(v, 0, Sx) = v.$$

Here and below a displayed equivalence between equations abbreviates transport along the indicated computation equations.

Lemma A.4 (The naming packet)**.** *For every term t, $\mathcal{R}$ proves $T(t)$. Moreover, the following fresh-variable naming rule is admissible:*

$$\frac{\Gamma, b = t \vdash F}{\Gamma \vdash F}, \qquad b \notin \mathrm{FV}(\Gamma, t, F). \tag{Name}$$

Proof. First prove $T(0)$ by RDI at witness 0. Its matrix is an equation, so the premise is progressive. Its successor row reduces by the two selector computations to $0 = 0$, independently of the induction hypothesis.

Next, with z a parameter, prove $T(Sz)$ by RDI at the permitted variable witness z. Its successor row reduces to $C(Sz, 0, Sz) = Sz$, and then to $Sz = Sz$, independently of its case assumption. QC applied to $v \mapsto T(v)$ now gives $Q_v T(v)$. Closure elimination at the arbitrary term t, using $T(0)$, gives $T(t)$. This last step is an application of QE, not substitution into the derivation of $T(v)$.

To prove Name, use the theorem $T(St)$ and open its witness at a fresh b. The resulting assumption is $Q_y A_{St}(b, y)$. Its zero row is a computation theorem, so specialization at 1 gives

$$C(1, St, C(St, 0, Sb)) = St.$$

The left side computes first to $C(St, 0, Sb)$ and then to Sb. Hence $Sb = St$, and (P) gives $b = t$. Cut this equation into the given derivation of F, and discharge the opened witness. The conclusion does not contain b. □

A.5 Recovery of every omitted introduction

Theorem A.5 (Restriction of witnesses to zero and variables)**.** *Subject to the equational predecessor lemma (P), $\mathcal{R}$ and the native scalar kernel derive exactly the same sequents. In particular, the combined witness-mode introduction may be restricted to witnesses that are either* 0 *or variables without changing the theory.*

Proof. It suffices to recover the native case introduction at an arbitrary term t; native equational progress was recovered by QP. Suppose

$$\Gamma, B(t,0) \vdash B(t,Sz) \tag{*}$$

with the usual freshness conditions. Name t by a fresh variable b. Inside the context $\Gamma, b = t$, introduce $D_{x,y}B(x,y)$ at the allowed witness b. Under its temporary assumption $B(b,0)$, full formula transport gives $B(t,0)$. Cut this into a weakened and injectively renamed copy of (*) to obtain $B(t,Sz)$. Full formula transport along $t = b$ gives $B(b,Sz)$. RDI at b proves the desired D formula. Name then removes $b = t$ and the fresh variable b.

This argument also recovers the omitted case introduction at literal 0 for an equation. Conversely, every rule of $\mathcal{R}$ is derived in the native kernel: case mode is its case introduction, and progressive mode is obtained from its equation-progress closure followed by case introduction. Induction on derivations in either direction completes the sequent equivalence. □

Corollary A.6 (Bi-interpretability and consistency strength)**.** *The ordinary realization of $\mathcal{R}$ is bi-interpretable with full first-order iPRA by the same interpretations as the native scalar kernel. Both number domains remain whole, equality remains actual equality, and both domain composites are the identity graph. In particular the theories are equiconsistent.*

Proof. The syntax and its ordinary reading have not changed, and the theorem gives equality of all contextual derivability relations, not merely equality of closed or arithmetically restricted consequences. Therefore the already established native bi-interpretation transfers unchanged. □

Relation to the pinned baseline. The README at the pinned commit `9f9545f5ffd9`, under "Further questions", item 2, conjectures the sufficiency of case-introduction witnesses 0 and variables, and correctly identifies the danger of silently using substitution into derivations of the restricted calculus. The proof here resolves that particular logical gap through QC, the explicit packet T, and the independently equational predecessor lemma. It does not assert any absolute minimality of the resulting checker, and its mathematical completeness claim is separate from any program-size measurement.

Auditable source interfaces. The pinned baseline's `NativeEq.lean` establishes the ordinary one-term equality interface and full formula transport; `NativeGate.lean` and `NativeLogic.lean` establish the standard logical encodings; and `NativeBiInterpretation.lean` states the native/iPRA comparison. The argument above reproves the transport and closure steps in the restricted calculus instead of importing them with their unrestricted proof-substitution lemmas. The predecessor construction must likewise be given an independent equational proof; the statement (P) is displayed to make this dependency explicit.

B Restriction to zero or the newest parameter

Let $\mathcal{R}_{\text{new}}$ be the restricted calculus of Section A, with variable witnesses further restricted to the newest numerical parameter of the current scope. The literal 0 remains available. At an empty scope only 0 is available. Introduction opens its step variable *after* selecting the witness, so the newest parameter here means the newest parameter before that step variable is opened.

This restriction is not invariant under arbitrary weakening of a derivation by a new numerical parameter. Accordingly, the direct structural proof from the preceding section cannot be reused. The following argument uses a protected translation instead. It assumes that the predecessor identity has been proved in the equational computation/progress fragment, whose derivations are invariant under numerical renaming; this is stronger than merely assuming an arbitrary derivation of that identity in the target calculus.

B.1 Hereditary derivability and uniform cut

Write $\Gamma \Vdash A$ if, for every injective renaming ρ of the numerical parameters into any larger numerical scope and every formula context Δ containing $\rho\Gamma$, there is an $\mathcal{R}_{\text{new}}$ derivation of ρA from Δ. This is a metatheoretic family of actual target derivations, not an additional rule or a second object language. It is closed by definition under injective renaming and formula-context weakening. Membership $A \in \Gamma$ implies $\Gamma \Vdash A$.

Lemma B.1 (Uniform cut)**.** *If Π is an actual target derivation of A from Γ, and $\Delta \Vdash B$ for every $B \in \Gamma$ in the same numerical scope, then there is an actual target derivation of A from Δ.*

Proof. Induct on Π. At a hypothesis use its given family at the identity renaming. At an eigenvariable rule, each replacement assumption has a new derivation in the enlarged numerical scope because its proof is hereditary. Add the temporary assumption as a hypothesis and apply the induction hypothesis to the subderivation. The numerical scope and the chosen witness of the original target rule have not changed, so a newest-variable witness is still admissible. No existing target proof is renamed under the new eigenvariable. Contexts are scoped: Δ has the same numerical scope as Γ, so it cannot already mention the new eigenparameter of Π. Thus its freshness condition is preserved as well. □

As a consequence, cut is admissible for $\Vdash$: if $\Gamma \Vdash A$ and $\Gamma, A \Vdash B$, then $\Gamma \Vdash B$. At each renamed extension, apply uniform cut to the second derivation using the hereditary family for the first. More generally a finite list of hereditary replacement assumptions can be cut simultaneously.

B.2 The logical and equational interfaces at the hereditary level

Computation, crossed transport, and QE preserve $\Vdash$. They introduce no new numerical parameter, and their conclusion is obtained by applying the same primitive rule to the available members of the proof families. Native DE also preserves $\Vdash$: at each renamed extension open the witness at the newest fresh parameter, and instantiate the hereditary family for its second premise at the correspondingly extended renaming. The original eigenparameter maps to this newly chosen parameter.

The closure rules QC and QP hold at the hereditary level. For QC, expand the proof through the neutral theorem $D_{r,w}(0 = 0)$ as in the preceding section. Its fresh witness b is newest, and hence

is admissible for the dummy-witness case introduction. The given step family is instantiated in the numerical scope containing b and a new step parameter z, rather than obtained by renaming a single target derivation. QP uses the literal witness 0 and is unchanged.

Full formula transport holds for hereditary proofs as well. The induction on the formula is exactly the explicit induction in the preceding section, with hereditary cut in place of ordinary cut. In the D case the witness that is reintroduced is the one just opened, so it is newest. In the Q case use hereditary QC. All equality and closure premises required below another binder are regenerated from their hereditary families. Thus

$$\Gamma \Vdash s = t, \quad \Gamma \Vdash H(s) \quad \Longrightarrow \quad \Gamma \Vdash H(t). \tag{HL}$$

This statement is uniform in all actual term assignments to free slots of H, as in the previous transport proof.

B.3 A hereditary naming packet

Retain $T(v) = D_{x,y}(C(y, v, C(v, 0, Sx)) = v)$. The theorem $T(0)$ has a derivation in every numerical scope by progressive introduction at 0. There is also a derivation of $Q_v T(v)$ in every numerical scope. To see this, *expand* QC instead of invoking it on a supposedly hereditary proof of $T(Sz)$: open the neutral theorem at a fresh b; introduce the dummy-witness D at b; and in its step premise, with z newly opened, prove $T(Sz)$ by case introduction at z. Here z really is newest, even though b has already been added. The successor row is a computation theorem. Eliminate the two dummy witnesses exactly as before.

This construction works afresh in every ambient numerical scope and formula context. Consequently $\varnothing \Vdash Q_v T(v)$ and $\varnothing \Vdash T(0)$. QE at an arbitrary actual term t then gives

$$\varnothing \Vdash T(t).$$

No claim that $T(Sz)$ has a hereditary direct-introduction proof was needed; such a claim would incorrectly allow z to cease being newest.

Opening $T(St)$ at a fresh newest b gives $b = t$ by computations and the predecessor identity. In the opened context, that equality has a hereditary proof: its derivation uses only the retained closure, QE, and equational derivations in the renaming-stable fragment. Therefore it can replace an assumed $b = t$ in an arbitrary target subderivation by uniform cut. The packet's DE then discharges b.

B.4 Translation of the native kernel

Theorem B.2. *The native scalar kernel and $\mathcal{R}_{\mathrm{new}}$ derive exactly the same sequents. Thus in the combined witness-mode rule it is sufficient to permit the literal* 0 *and the newest numerical parameter, and no other witness.*

Proof. Induct on a native-kernel derivation to prove the stronger assertion $\Gamma \Vdash A$. All cases except native case introduction follow from the hereditary interfaces just established.

Consider a case introduction at an arbitrary term t, whose premise is $\Gamma, B(t, 0) \vdash B(t, Sz)$. Its induction hypothesis is a hereditary family. To prove the conclusion at a chosen renaming ρ and context extension Δ, open the hereditary packet $T(S\rho t)$ at the newest fresh parameter b. It supplies the hereditarily derivable equation $b = \rho t$.

Temporarily retain $b = \rho t$ as an assumption in addition to Δ weakened to the scope containing b. Construct a target case introduction of the desired D formula at this newest witness b. In its premise, under the temporary assumption $B(b, 0)$ and a newly opened step parameter, hereditary transport gives $B(\rho t, 0)$. Instantiate the source premise's hereditary family at the injection sending the source step parameter to this new step parameter. Hereditary cut supplies its missing zero-row assumption, yielding $B(\rho t, Sz)$. Hereditary transport backwards gives $B(b, Sz)$. Take the member of this step family at the identity renaming and apply the primitive target introduction at b. Finally use uniform cut to replace the retained assumption $b = \rho t$ by its hereditary proof from the opened packet; the other assumptions are supplied by membership in the opened context. Then discharge the packet. The resulting formula is independent of b.

The construction was performed for every ρ, Δ, so it proves the required hereditary conclusion. The reverse implication is immediate: every target primitive rule is already a derived rule of the native kernel. Taking the identity member of the hereditary family completes the comparison. □

The result removes the need to supply an arbitrary term or a variable index as an introduction witness. A checker need only choose between two fixed witnesses, 0 and the newest parameter, rejecting the second choice at an empty scope. This is a statement about the available witness data, not by itself a claim that a particular serialization or complete checker is smaller.

B.5 A total two-choice interface

For an implementation it is simpler to make both choices total. Let $\beta \in \{0, 1\}$ be the only introduction datum, and in a numerical scope of size n put

$$t_{n,\beta} := \begin{cases} 0, & \beta = 0 \text{ or } n = 0, \\ b_{\text{new}}, & \beta = 1 \text{ and } n > 0, \end{cases} \qquad \eta_{B,\beta} := \begin{cases} z, & B \text{ is an equation and } \beta = 0, \\ 0, & \text{otherwise.} \end{cases}$$

The introduction is the single schema

$$\frac{\Gamma, B(t_{n,\beta}, \eta_{B,\beta}) \vdash B(t_{n,\beta}, Sz)}{\Gamma \vdash D_{x,y} B(x, y)}. \tag{Bit-DI}$$

The mode is selected by the bit, not by inspecting the chosen term. Thus the choice $\beta = 1$ in an empty scope is case introduction at 0, including when the matrix is an equation.

Corollary B.3 (One-bit introduction)**.** *Replacing RDI by Bit-DI gives exactly the native scalar kernel's derivable sequents.*

Proof. Every rule of $\mathcal{R}_{\text{new}}$ is an instance of Bit-DI. Every instance of Bit-DI is either ordinary case introduction or equational progress at witness 0, hence is derived in the native kernel. The comparison theorem sandwiches the new calculus between two calculi with the same derivability relation. □

At the certificate level one may require a literal Boolean datum, or define a total projection of an arbitrary JSON value onto a bit. Either is a serialization choice that must be stated explicitly. The mathematical certificate field no longer represents a term or a variable index. In particular its validation never needs to traverse a proposed witness term. Replacing the empty-scope rejection by the specified zero fallback is sound by the corollary; it is not an unchecked exception to the syntax.

C Direct transfer to the five-rule clause checker

The delivered checker uses the raw Eq/D clause presentation, rather than formulas with a primitive Q. The newest-witness restriction must therefore be proved directly for that presentation. An unrestricted native-to-clause translation can insert numerical parameters, so it cannot simply be assumed to preserve the newest-witness condition.

C.1 The clause interface

Formulas are equations and $D_{x,y}B(x,y)$. A retained clause

$$[A; y.B(y)]$$

has ordinary reading $A \to \forall y B(y)$. An ordinary assumption F is represented by $\mathsf{a}(F) := [0 = 0; y.F]$, with y unused. The five primitive families are Refl, the two-case Eval, Use, Elim, and Bit-Intro. Use takes a proof of the antecedent of a retained clause and returns its body at a supplied term. Elim opens a proved $D_{x,y}B$ at a fresh parameter b by retaining

$$[B(b,0); y.B(b,y)]$$

in its branch. Bit-Intro adds the ordinary-assumption clause $\mathsf{a}(B(t_{n,\beta}, \eta_{B,\beta}))$ in its step branch and checks the conclusion $B(t_{n,\beta}, Sz)$, with the selectors from Bit-DI. All terms are raw; no normalization is implicit.

Refl and Eval alone derive both iterator computations and the whole equational interface. For example, the zero case of Eval is

$$s = t, \quad I(0, s; u.h) = r \quad \Longrightarrow \quad r = t.$$

Two uses, through the reflexive frame $I(0, s; u.h)$, derive the root joining rule $s = t,\ s = r \Longrightarrow r = t$. The successor case applied to a one-step iterator then gives congruence in an arbitrary term template. All these derivations are uniform in the numerical scope and use no introduction or elimination of D. They therefore remain valid when regenerated under any numerical renaming.

Define the closure abbreviation

$$q_y A(y) := D_{x,y} A(y), \qquad x \text{ unused.}$$

For the total bit interface, case introduction of qA is a direct Bit-Intro with $\beta = 1$: the witness is unused, and the empty-scope fallback still selects cases. Equational progress introduction of qE is direct Bit-Intro with $\beta = 0$. Closure elimination is an Elim of qA, followed by Use of the resulting clause $[A(0); y.A(y)]$. These observations implement the Q interface without a primitive Q constructor. Hereditary premises, defined next, supply any proofs needed below the fresh dummy witness.

C.2 Hereditary derivations with clause contexts

Write $K \Vdash_c F$ if every injective numerical renaming of K, F into any larger finite numerical scope is derivable in the bit clause calculus from every clause-context extension of the renamed K. Renaming acts capture-avoidably on both the antecedent and the bound body of each clause. The extension

retains clauses, including their entire antecedent/body data; a retained clause is not replaced by the assertion that its antecedent or one instance of its body is true.

The cut principle needed here concerns *ordinary-assumption* clauses only:

$$K, \mathsf{a}(A) \vdash_c F, \qquad K \Vdash_c A \quad \Longrightarrow \quad K \vdash_c F. \tag{UC_c}$$

It also holds when the other retained clauses are embedded in a larger target context. Prove it by induction on the actual first derivation. When Use selects the distinguished clause $\mathsf{a}(A)$, replace that use by the available member of the hereditary family for A; its antecedent proof of $0 = 0$ is no longer needed. Every other clause use keeps the same clause and applies the induction hypothesis to its antecedent proof. Elim and Intro add their own clauses in both contexts; the replacement family is regenerated in the enlarged numerical scope. Thus no arbitrary clause is cut or semantically reconstructed, and no newest-sensitive proof is renamed. The same argument gives cut for $\Vdash_c$ at an ordinary-assumption clause.

Refl, Eval, and Use preserve $\Vdash_c$. For Use the selected clause is a member of the renamed context, and the hereditary premise supplies its antecedent. Elim preserves $\Vdash_c$ by extending the source injection to send its eigenparameter to the newest fresh target parameter; its new clause is retained literally in the target branch.

Full formula transport is proved by induction on Eq/D formulas at the hereditary level. The equation case is the equational interface above. For $D_{x,y}B(s, x, y)$, open its witness at the newest b and retain the clause $[B(s, b, 0); y.B(s, b, y)]$. Introduce the target D at b with bit 1. From its temporary ordinary assumption $B(t, b, 0)$, the induction hypothesis transports backwards to $B(s, b, 0)$. Use of the retained clause at Sz gives $B(s, b, Sz)$; forward transport gives $B(t, b, Sz)$. Hereditary ordinary cut composes these operations. The result is independent of b, so Elim discharges it. This proves

$$K \Vdash_c s = t, \quad K \Vdash_c F(s) \quad \Longrightarrow \quad K \Vdash_c F(t). \tag{HL_c}$$

The induction is uniform in actual term assignments to the other free slots. No arbitrary term is substituted into a target proof.

C.3 The packet and the completeness induction

The same raw equation packet works:

$$T(v) := D_{x,y}(C(y, v, C(v, 0, Sx)) = v).$$

At any ambient scope, $T(0)$ is obtained with bit 0 by a progressive introduction whose successor row is a computation theorem. To construct $q_v T(v)$, use bit 1 on the outer dummy witness. In its step branch the parameter z is newest; construct $T(Sz)$ with bit 1, choosing exactly that z. Its successor row again computes to reflexivity. This construction is repeated in each ambient scope. It does not assume that a direct proof of $T(Sz)$ remains valid after z ceases to be newest.

Elim followed by Use of the resulting closure clause gives $K \Vdash_c T(t)$ for every term t. Opening $T(St)$ at the newest b and using its clause at 1 gives $Sb = St$. The independently equational predecessor identity gives $b = t$. That equality has a hereditary proof in the opened clause context, because the calculation uses only retained-clause Use and the renaming-stable equational fragment.

Now induct on an unrestricted clause-calculus derivation, proving hereditary derivability of its conclusion. Refl, Eval, Use and Elim were handled above. A progressive introduction at witness 0 is directly bit 0. For a case introduction at any term t, open $T(St)$ at a newest fresh b. Temporarily

retain $\mathsf{a}(b = t)$ and introduce the desired D with bit 1. In its step branch, hereditary transport changes the temporary assumption $B(b, 0)$ into $B(t, 0)$. Instantiate the induction hypothesis for the original source premise in the enlarged scope: the original source variables keep their chosen injection, the source step parameter maps to the newest target step parameter, and the alias parameter b is skipped by this injection. Ordinary hereditary cut supplies the zero-row assumption, and transport backwards gives $B(b, Sz)$.

After this introduction, UC_c replaces the retained ordinary assumption $b = t$ by its hereditary proof from the opened packet. Elim discharges b. The argument applies at every numerical renaming and clause-context extension, so it proves the hereditary induction claim.

Conversely, every bit introduction is either ordinary cases or equational progress. The only extra case relative to the baseline's witness-tested combined rule is bit 1 at an empty scope with an equation matrix: this is case introduction at 0. It is derived in the unrestricted raw clause calculus by using the nonliteral alias $w = I(0, 0; u.0)$ as its witness and transporting along $w = 0$. Since w is not the literal 0, the baseline rule selects cases. Thus no semantic strengthening occurs at the empty-scope fallback.

Theorem C.1 (Five-rule bit clause calculus)**.** *The five-rule raw Eq/D clause calculus with Bit-Intro derives exactly the same sequents from the same clause contexts as the unrestricted clause calculus. In particular it has the same ordinary realization, and hence the same full first-order bi-interpretation with iPRA.*

The proof is a direct transformation into the restricted clause calculus. It does not rely on an intermediate proof translation preserving which numerical parameter happens to be newest.

D A predecessor from equations and equation induction alone

This appendix supplies the arithmetic fact needed by the restricted-witness theorem: successor injectivity follows from a purely equational fragment, without case introduction for D. The construction of the predecessor is inherited from the repository's scalar bootstrap. The contribution of this appendix is its purely equational organization, in particular the replacement of case splits made under an induction hypothesis by unconditional finite identities proved in advance. The argument below uses no numerical coverage axiom, no logical disjunction, and no successor injectivity. After equation induction is made available through the interface below, the arithmetic argument itself has no D formulas.

D.1 Resources and a finite-value lemma

The available terms are variables, 0, S, and the iterator $I(n, a; u.h)$, whose only bound variable is u in h. We use

$$I(0, a; u.h) = a, \tag{1}$$

$$I(Sn, a; u.h) = h[I(n, a; u.h)/u], \tag{2}$$

ordinary equality reasoning, and induction on a single equation. In the native calculus the latter is obtained from QI by progress and QE:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}, \qquad z \notin \mathrm{FV}(\Gamma),$$

where E is a literal equation and all its other parameters stay fixed. In a combined-introduction presentation, QI by progress is obtained by introducing an equation matrix with unused witness at 0 and then opening that binder. The equational argument itself needs only the displayed equation-induction interface.

Instantiation in the equational fragment. Capture-avoiding substitution of arbitrary terms for numerical parameters is admissible in this equational fragment. This follows by induction on an equational derivation: computation and transport commute with substitution, and an induction eigenvariable is first renamed fresh for the terms being substituted. In the combined-rule implementation of equation induction, the witness remains the literal 0 and the matrix remains an equation. Thus this argument never substitutes into a case introduction at a variable witness. Completed equational identities may therefore be instantiated at arbitrary terms without presupposing a substitution theorem for the full restricted calculus.

Ignoring the induction hypothesis gives *equational cases*: proofs of $E(0)$ and $E(Sz)$ give $E(t)$ for every term t. This is not a case rule for arbitrary formulas. It is a derived rule only for equations.

Write $1 = S0$, with larger numerals defined similarly, and put

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad N(n) = I(n, 0; u.Z(u)).$$

The update variable is absent from the body of C, so computation gives

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b, \tag{3}$$

$$Z(0) = 1, \qquad Z(Sn) = 0, \tag{4}$$

$$N(0) = 0, \qquad N(Sn) = Z(N(n)). \tag{5}$$

Finite-value transfer. Let $F(v)$ be an equation, and suppose that $F(0)$ and $F(1)$ have been proved over a fixed side context Γ. Then $F(Z(t))$ and $F(N(t))$ are derivable over Γ for every term t. The placeholder v is fresh for Γ; all other parameters of F remain fixed.

For the first conclusion, use equational cases on a fresh variable x with equation $F(Z(x))$. Its zero case is $F(1)$ and its successor case is $F(0)$, by (4). For the second, use equational cases on x with equation $F(N(x))$. Its zero case is $F(0)$; its successor case is $F(Z(N(x)))$, already provided by the first conclusion. Finally instantiate the completed equations at $x = t$.

In subsequent uses of this lemma, the two equations $F(0)$ and $F(1)$ are unconditional computations. We never infer an equation by splitting on $N(t) = 0$ or $N(t) = 1$ while a side assumption depends on that value.

D.2 Addition and parity identities

Define

$$a + b = I(b, a; u.Su), \qquad 2a = a + a, \qquad 3a = a + a + a.$$

Thus $a + 0 = a$ and $a + Sb = S(a + b)$. Equation induction on b gives $0 + b = b$ and $Sa + b = S(a + b)$. Another induction on b proves commutativity: the successor step is

$$a + Sb = S(a + b) = S(b + a) = Sb + a.$$

Induction on the last summand proves associativity. We shall reorder finite sums using these completed identities. In particular, $a + 1 = Sa$, $2(Sa) = 2a + 2$, and $3(Sa) = 3a + 3$.

The instances at $v = 0, 1$ of

$$v + Z(v) = 1, \qquad Z(Z(v)) = v$$

compute directly. Finite-value transfer therefore gives

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n). \tag{6}$$

Two successor computations in (5) now give

$$N(n + 2) = N(n). \tag{7}$$

An equation induction on n proves $N(2n) = 0$: the successor step reduces $N(2n + 2)$ to $N(2n)$. Consequently $N(2n + 1) = Z(0) = 1$. An independent equation induction proves $N(3n) = N(n)$: after increasing n, the left argument increases by three, so period two reduces the new left value to $Z(N(3n))$, and the induction equation changes this to $Z(N(n)) = N(Sn)$. We have proved

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{8}$$

No assertion that every number is even or odd has been used.

D.3 A three-state update and a ramp

Put

$$j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)), \qquad G(n) = I(n, 0; u.u + M(u) + 1).$$

Finite computations give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0. \tag{9}$$

We need these values and $M(Sn) = j(M(n))$; we do not assume a remainder theorem for M.

For arbitrary parameter g, the following equations hold whenever $b = N(v)$:

$$M(g + b + 1) = C\big(b, j(M(g)), j(j(M(g)))\big), \tag{10}$$

$$C\big(b, j(b), j(j(b))\big) = Z(b). \tag{11}$$

To prove (10), apply finite-value transfer to the equation with placeholder b. At $b = 0$ it is $M(g + 1) = j(M(g))$, and at $b = 1$ it is $M(g + 2) = j(j(M(g)))$, both computation identities. For (11), the zero instance is $j(0) = Z(0) = 1$, and the one instance is $j(j(1)) = j(2) = Z(1) = 0$. These are again computations. These two identities are proved *before* the next induction.

We claim

$$M(G(n)) = N(n). \tag{12}$$

The zero instance is immediate. In the successor step retain the single induction equation $M(G(n)) = N(n)$ and calculate

$$\begin{aligned} M(G(Sn)) &= M\big(G(n) + M(G(n)) + 1\big) \\ &= M\big(G(n) + N(n) + 1\big) \\ &= C\big(N(n), j(M(G(n))), j(j(M(G(n))))\big) \\ &= C\big(N(n), j(N(n)), j(j(N(n)))\big) \\ &= Z(N(n)) = N(Sn). \end{aligned}$$

The second and fourth lines use the induction equation; the third and fifth use the two unconditional identities just proved. In particular, there is no case split under the induction hypothesis.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3. \tag{13}$$

The second equation follows from the first applied twice and $N(n) + Z(N(n)) = 1$. Equation induction gives

$$G(2n) = 3n. \tag{14}$$

For the successor step, use $G(2n+2) = G(2n) + 3$.

A separate equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{15}$$

Indeed, (13) gives $G(x+3) = G(x) + N(x) + 4$. Assuming the induction equation and using $N(3n) = N(n)$, the new left side is

$$G(3Sn) = G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4.$$

The new right side is

$$3G(Sn) + N(Sn) = 3G(n) + 3N(n) + 3 + Z(N(n)).$$

The two expressions are equal by $N(n) + Z(N(n)) = 1$ and the additive identities. The base is $0 = 0$. Again, no conditional case distinction occurs inside the induction.

D.4 The factor map

Define

$$H(u) = C(N(u), G(u), u).$$

Then

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{16}$$

The first two equations follow immediately from (8), (14), and the two selector computations. For the third, first prove by finite-value transfer the unconditional equation

$$C(b, 3a+b, 3c) = 3C(b, a, c), \qquad b = N(v). \tag{17}$$

At $b = 0$ both sides are $3a$, and at $b = 1$ both sides are $3c$. Now substitute $v = u$, $a = G(u)$, and $c = u$ into this completed identity:

$$\begin{aligned} H(3u) &= C(N(u), 3G(u) + N(u), 3u) \\ &= 3C(N(u), G(u), u) = 3H(u). \end{aligned}$$

This argument does not represent an arbitrary number as $2v$ or $2v+1$.

D.5 A comparator

For a fixed update term h, whose parameters do not include the variable being inducted on, equation induction proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{18}$$

At $y = 0$ both sides are $h(a)$. In the step, the two successor computations apply the same fixed h to the induction equation. Here $I(y, a; h)$ abbreviates $I(y, a; u.h(u))$; it is a term abbreviation, not quantification over functions.

Using $H(3u) = 3H(u)$, another equation induction on y, with a fixed, gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{19}$$

The base is $3a = 3a$; the step rewrites $H(I(y, 3a; H))$ to $H(3I(y, a; H)) = 3H(I(y, a; H))$. Similarly, $H(1) = 1$ gives

$$I(y, 1; H) = 1. \tag{20}$$

Set

$$A(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, A(x); H)\big).$$

We obtain the three comparison equations

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{21}$$

The first uses $A(0) = 1$, (20), and $N(1) = 1$. The second uses $A(Sx) = 2A(x)$ and $N(2u) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2A(x); H)\big) \\ &= N\big(I(y, H(2A(x)); H)\big) \\ &= N\big(I(y, 3A(x); H)\big) \\ &= N\big(3I(y, A(x); H)\big) \\ &= N\big(I(y, A(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2u) = 3u$, orbit scaling, and $N(3u) = N(u)$. Thus (21) holds internally at all arguments. The intended interpretation of $L(x, y)$ as the indicator of $x \leq y$ is not a premise.

D.6 Saturation and predecessor

Define the following open terms, with m a fixed parameter:

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

Equational cases on a fresh variable e prove

$$C(e, Sa, Sb) = SC(e, a, b). \tag{22}$$

At $e = 0$ both sides are Sa, and at $e = Sz$ both are Sb. Using (21) and this selector equation gives

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (23) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) \\ &= C(L(0, m), 0, S0) = C(1, 0, S0) = S0. \qquad (24) \end{aligned}$$

Equation induction on t, with m fixed, now proves

$$W(St, Sm) = SW(t, m). \qquad (25)$$

For $t = 0$, the left side is $B_{Sm}(0) = S0$, as required. The step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the active induction equation.

Two more equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (26)$$

For the first, the base is $L(S0, 0) = 0$ and the step reduces $L(SSn, Sn)$ to $L(Sn, n)$. For the second, the base is $W(0, 0) = 0$, and the step uses the completed identity (25) at $t = m = n$: $W(Sn, Sn) = SW(n, n) = Sn$. This substitution is into an already proved identity, and does not change a parameter of an active induction hypothesis. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (27)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (28)$$

Its zero equation is a computation. For its successor equation put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The third equation in (21) gives the pointwise identity

$$C(L(SSu, Sm), u, Su) = C(L(Su, m), u, Su) = B_m(u).$$

Equation induction on t, with m fixed, proves $V(t, m) = W(t, m)$. The base is $0 = 0$. In the step, first apply the pointwise update identity at $u = V(t, m)$ and then the induction equation, obtaining $B_m(V(t, m)) = B_m(W(t, m))$. Thus no primitive extensionality rule for iterator bodies is being assumed. From (27) we conclude

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.} \qquad (29)$$

Applying equality congruence for P to $Sx = Sy$ and using (29) gives $x = y$.

Every abbreviation in this appendix expands to a finite term over $0, S, I$. Every induction invariant is one literal equation. Equality reflection, pairing, general primitive-recursion compilation, numerical coverage, and logical case analysis were not used to obtain predecessor or successor injectivity.

D.7 Provenance and verification status

The arithmetic construction, including the factor map and the saturated iteration, comes from `Research/certified_zero/reference/scalar_predecessor.tex` in the user's ConciseIPRA project, read at commit `9f9545f5ffd9e0392d4a36639503aca12383450a`. The repository URL is

ConciseIPRA at the pinned commit

The existing source explicitly allowed reconstructed numerical cases. The present appendix strengthens its dependency statement by giving equation-only proofs of the needed finite identities, particularly (10), (11), and (17). These identities remove the circularity that would arise if a proof of restricted logical introduction already assumed unrestricted logical case distinction.

This appendix is a written internal derivation. It does not claim a newly compiled Lean formalization of the equational dependency refinement or of the restricted-witness theorem. The repository's preexisting Lean formalization should not be presented as machine verification of this new dependency claim without the corresponding new formal proof.

E Archive map and reproduction

Path	Purpose
`ipra_parameter_witness.tex`, PDF	Main report and complete new proof appendices.
`paper/`	LaTeX sources for the restriction, protected translation and arithmetic.
`checker/checker.py`	Standalone five-family bit-witness checker.
`checker/reference/`	Exact pinned unrestricted checker controls.
`checker/reproduce.py`	Exact source metrics and certificate checks.
`code/check_scalar_arithmetic.py`	Finite evaluation from iterator definitions.
`checker/fixtures/`, `evidence/`	Checked certificates, tests, metrics and validation logs.
`reference/`	Attributed baseline bridge sources and inspected Lean endpoints.
`PROVENANCE.json`	Exact repository paths, commit and source hashes.
`CLAIMS.json`	Claims, dependencies and explicit limitations.
`reproduce.py`	Re-run executable validation and measurements.
`build.py`	Rebuild the PDF with installed pdfLaTeX.

Run `python3 reproduce.py` from the extracted directory. Run `python3 build.py` to rebuild the PDF; a normal LaTeX installation with the packages named in the source is required. The executable checker and validation scripts use only the Python standard library. Measured normalized sizes should be reproduced with CPython 3.12.14, since `ast.unparse` is version-dependent.

References

[1] ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, repository snapshot, commit `9f9545f5ffd9` (full hash on the title page), 12 November 2026. Pinned repository snapshot.

[2] Same snapshot, *README*, especially "Further questions", item 2, and the raw clause / two-case evaluation entries. The exact retrieved README is included in the source provenance.

[3] Same snapshot, `Research/rootfree_clausal` and `Research/reduced_clause_calculus`, existing two-case evaluation and raw clause presentation. The exact matched control path is recorded in `PROVENANCE.json`.

[4] Same snapshot, `Lean/ConciseIPRA/NativeEval.lean`, `NativeClausal.lean` and the cited clause-calculus successors.

[5] Same snapshot, `Lean/ConciseIPRA/NativeBiInterpretation.lean`. Pinned Lean bi-interpretation endpoint.

[6] Same snapshot, `Research/certified_zero/reference/`: `inherited_choice_basis.tex`, `inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, `primitive_recursion.tex`. These are included unchanged under their repository license.